

# Modelling Postgres Performance Degradation on Burstable Cloud Instances

Goh Chun Lin

.NET Foundation Member

June 18, 2026

# The Appeal of Burstable Instances

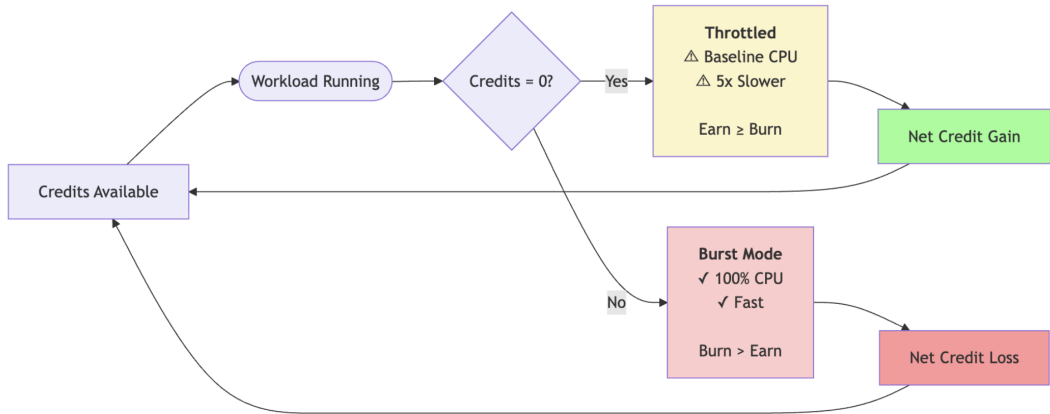
## Cost-Effective Cloud Computing

- Azure B-series (Burstable PostgreSQL)
- Similar offerings: AWS T-series, GCP E2 (burstable)
- 20–60% cheaper than fixed-performance instances

## The Core Question

Perfect for variable workloads... right?

# Token Bucket Algorithm



# The CPU Credit Model

## How it works:

- ① Earn credits continuously over time
- ② Burn credits when using CPU above baseline
- ③ Burst to 100% CPU when credits available
- ④ Throttle to baseline when depleted

## Example Blueprint: Azure B2ms

- Base CPU Performance of VM: 30%
- Earns 36 credits/hour, burns 2 credits/min at burst

# Azure B-series V1 CPU Burst

| Size Name      | Base CPU Performance of VM (%) <sup>1</sup> | Initial Credits (Qty.) | Credits banked/hour (Qty.) | Max Banked Credits (Qty.) |
|----------------|---------------------------------------------|------------------------|----------------------------|---------------------------|
| Standard_B1ls  | 5%                                          | 30                     | 3                          | 72                        |
| Standard_B1s   | 10%                                         | 30                     | 6                          | 144                       |
| Standard_B1ms  | 20%                                         | 30                     | 12                         | 288                       |
| Standard_B2s   | 20%                                         | 60                     | 24                         | 576                       |
| Standard_B2ms  | 30%                                         | 60                     | 36                         | 864                       |
| Standard_B4ms  | 22.5%                                       | 120                    | 54                         | 1,296                     |
| Standard_B8ms  | 17%                                         | 240                    | 81                         | 1994                      |
| Standard_B12ms | 17%                                         | 360                    | 121                        | 2,908                     |
| Standard_B16ms | 17%                                         | 480                    | 162                        | 3,888                     |
| Standard_B20ms | 17%                                         | 600                    | 202                        | 4,867                     |

Reference Documentation:



# The Hidden Danger

**Not a crash. Not an error. Just... slowness.**

When CPU credits deplete:

- Database throttles to baseline CPU;
- Query time increases significantly (5x minimum, often worse);
- Postgres does not know it's being throttled;
- Slow queries hold connections longer.

## Result

Connection pool exhaustion.

# The Cascading Failure

**Watch how one problem triggers the next:**

When connections held longer:

- Connection pool saturates;
- New requests queue;
- P99 latency spikes and app layer times out;
- max\_connections limit hit;
- PostgreSQL rejects connections.

## Result

At this point, you are experiencing a severe production outage.

# The Restart Loop

**But wait - it gets worse:**

When PostgreSQL rejects connection:

- Health checks fail;
- Orchestrator on Azure restarts the database;
- Boot sequence burns remaining CPU credits;
- Comes back up already slow and immediately saturates again;
- Manual intervention required to break the loop.

# The Challenge: Incomplete Information

**You can calculate credit rates. You can monitor credit balance. But predicting interactions requires testing.**

| Over-provision                                          | Load test all configs                             | Simulate                                                       |
|---------------------------------------------------------|---------------------------------------------------|----------------------------------------------------------------|
| Skip the B-series entirely and pay for General Purpose. | Trial by fire. Test every possible configuration. | Use the math to model the traffic. Test 50 scenarios in hours. |
| ✓ Safe                                                  | ✓ Accurate                                        | ✓ Good enough                                                  |
| × Expensive at scale                                    | × Weeks of work                                   | ✓ Fast                                                         |

# Simulation Approach

**Models credit mechanics, pooling modes, and queueing behavior**

**Simulation is a filter, not a total replacement for reality.**

- Reduces load testing scope (test 3 finalists, not 50 configs)

**Simulation finds candidates. Load testing validates the finalists.**

# Why Discrete Event Simulation (DES)?

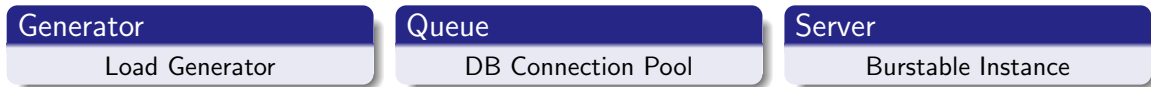
## Definition

**DES:** Models systems by following each event step by step.

**DES models pooling, CPU credits, queueing in burstable instances.**

# Modelling as Discrete Event Simulation

**Burstable instances are not magic. They follow a strict mathematical model called the Token Bucket Algorithm.**



# What is SNA?

A lightweight open-source library for Discrete Event Simulations in C# and .NET 10.

**Available to model Azure Database for PostgreSQL B-series with M/M/c/K:**

- Deterministic logic for stochastic inputs;
- Configurable baseline/earn rates (adapts to any B-series specs);
- Validated against queueing theory predictions.

Project repo:

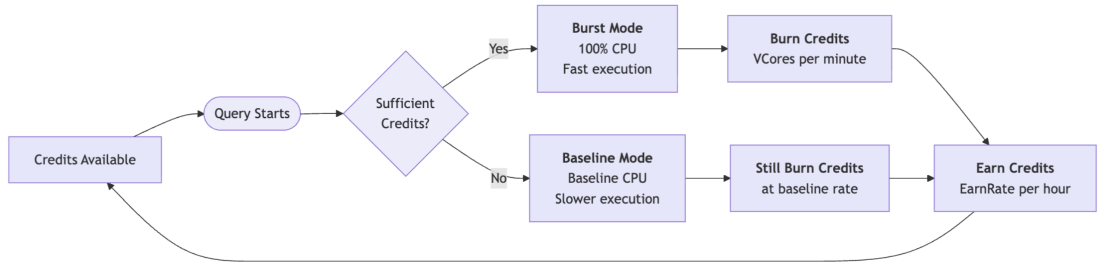


# CPU Credits Simulation

## Question:

Where is the point where specific workload transitions from "Bursting" to "Death Spiral"?

# Credit Mechanics

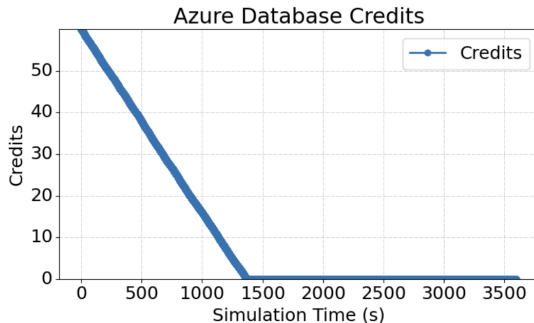


# Credit Depletion Timeline

## What we're seeing:

- Credits start at 60;
- Linear depletion as burst usage exceeds earn rate;
- Credits hit zero at 1,300 seconds;
- **This is your burst tolerance window: ~20 minutes.**

*\*Note: Starting credits affect timeline. Tune this parameter to match your environment.*



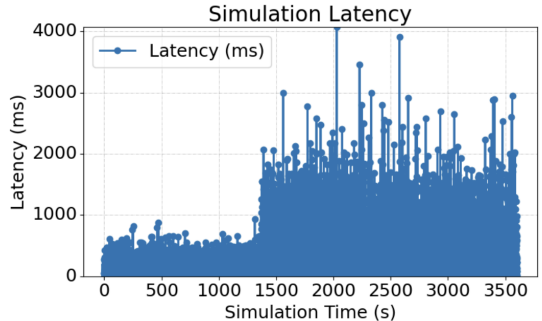
# The Performance Cliff

## What happens when credits hit zero:

- Simulation shows 5x degradation for CPU-bound queries;
- Production cascading effects often worse.

### Takeaway

This is why "database is up" does not mean database is working.



## Demo 2: Pool Size Optimization

### Question:

How big should my connection pool be?

## Demo 2: Testing Pool Sizes

### Testing common assumptions:

| Pool Size | Common Industry Rationale        |
|-----------|----------------------------------|
| 5         | "Too small?"                     |
| 10        | "Rule of thumb: use 10"          |
| 20        | "2x CPU cores (B2ms has 2 vCPU)" |
| 50, 100   | "Just set it high to be safe"    |

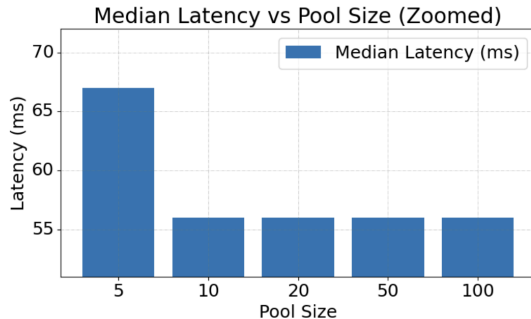
**Parameters:** Session pooling configuration, 50 requests/sec continuous steady-state load.

**Objective:** Isolate the minimum pool footprint within 5% of optimal latency.

## Demo 2: Results

### Key findings:

- **Pool Size 5:** ~67ms (queueing visible)
- **Pool Size 10:** ~57ms (optimal for steady load)
- **Pool Size 20–100:** ~56ms (no latency gain, but handles micro-bursts)



## Industry-standard credit model:

- **Azure B-series**, AWS T-series, GCP E2, Oracle burstable
- All use: earn credits → burn when bursting → throttle when depleted

**Minor differences:** Initial credits, unlimited mode, baseline %

**Same approach works across all cloud providers.**

# When Simulation Is Worth It

**Startup (10 instances):** Just guess (config changes are fast)

**Enterprise (50+ instances):** Simulate (saves \$50k/year vs over-provisioning)

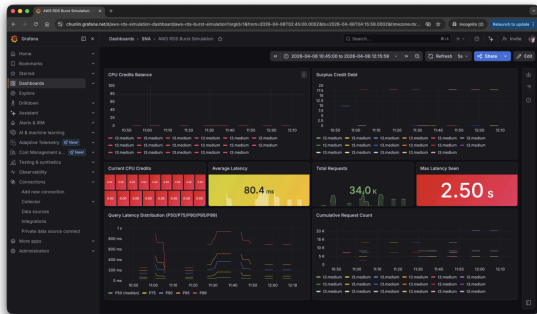
**This model covers:** CPU credits, connection pooling, queueing

**Does not model:** IOPS throttling, network throttling, storage latency

# One More Thing

Observability

# Observability: OpenTelemetry Integration



## What you're seeing:

- **Credits depleted:** All instances at 0.00 (red boxes)
- **Performance cliff:** Max latency exploded to 2.5 seconds
- **The problem made visible:** Average 80ms looks fine, but p99 catastrophic

**SNA exports OpenTelemetry metrics for real-time monitoring**

# From Guessing to Knowing

| Before Simulation   | After Simulation           |
|---------------------|----------------------------|
| Guess pool size     | Calculated optimal size    |
| Test in production  | Test in simulation first   |
| Risk of downtime    | Identified failure points  |
| Days to tune        | 1 minute to narrow options |
| Over-provision 2-5x | Data-driven right-sizing   |

**Thank You**

**GitHub Repository:**

[github.com/gcl-team/SNA](https://github.com/gcl-team/SNA)

**Contact:**

Goh Chun Lin

[linkedin.com/in/gohchunlin](https://linkedin.com/in/gohchunlin)